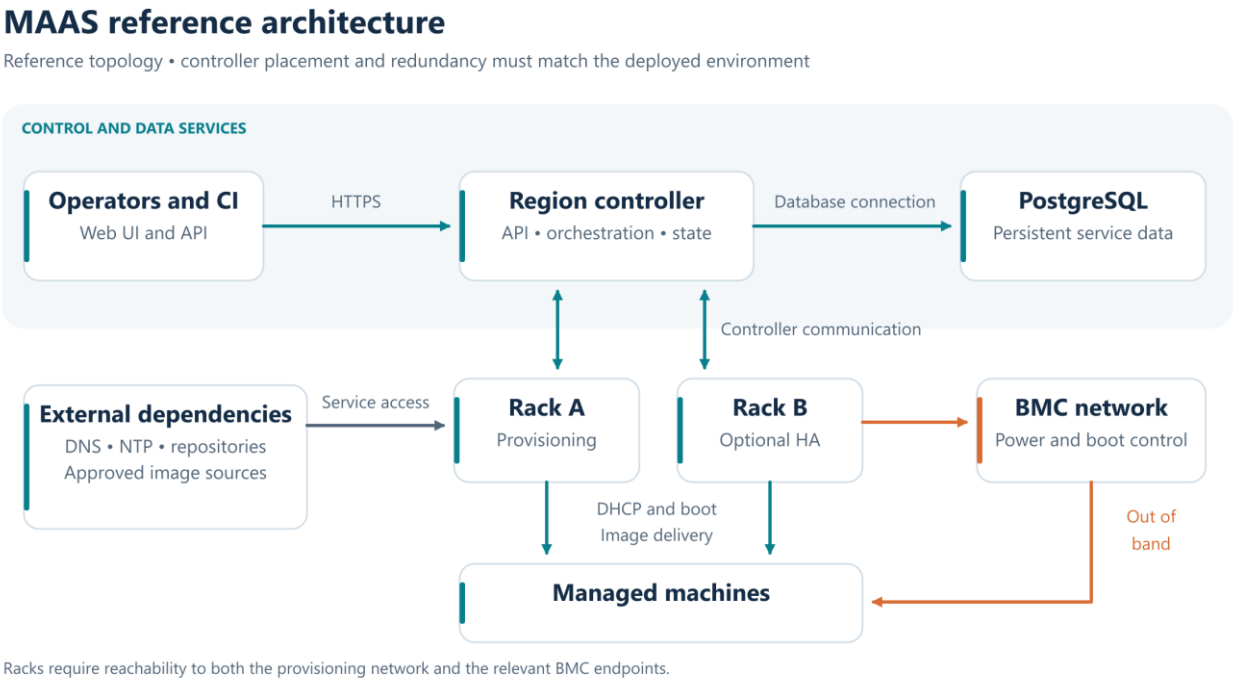
MAAS on Ubuntu

Operations and Handover Guide

Bare metal provisioning
Image engineering
Service recovery



Reference architecture for a managed bare metal provisioning service

This guide equips infrastructure engineers to operate the MAAS service, qualify server deployments and recover the control plane. Operational acceptance requires a completed production register and the evidence defined in the handover tests.

Edition 1.0 | 24 October 2026
Prepared for Infrastructure Operations

Prepared by **Aref Shahsavari**

How to use this guide

Start with the production register, then validate the service architecture and dependencies. Use the runbooks for daily work and the acceptance tests for ownership transfer. Commands are Ubuntu Bash examples; replace sample values and check the installed API before executing state changes.

Reference diagrams describe logical designs. Complete the production register with approved site values, owners and evidence. The operational procedures apply to MAAS 3.x concepts; release dependent details are explicitly qualified and linked to primary documentation.

Contents

03 Service ownership and configuration register
04 Architecture and service boundaries
05 Release baseline and rebuild procedure
06 Network design and address allocation
07 DHCP DNS time and firewall dependencies
08 Machine lifecycle and operational gates
09 Commissioning and hardware qualification
10 Storage layout and RAID operations
11 Deployment runbook and CLI access
12 Cloud init baseline and validation
13 Standard images and repository operations
14 Custom image engineering and promotion
15 Build and publish an Ubuntu custom image
16 Access control and security operations
17 Automation and configuration management
18 Monitoring and service objectives
19 Troubleshooting by failure stage
20 Backup design and capture procedure
21 Disaster recovery and restore validation
22 Availability and controlled upgrades
23 Routine operations and capacity management
24 Acceptance tests and operational signoff
25 Operator quick reference and terminology
26 Technical references

Operating discipline: confirm identity and state, preserve evidence, apply the smallest justified change, and verify the outcome. Installation success alone does not establish that a machine is ready for its workload.

Service ownership and configuration register

Complete this register with verified production values before operational acceptance. Store credentials in the approved secrets service and record references here. Maintain the register alongside the configuration repository after every material change.

Service attribute	Production record
Service name and environment	Record service identifier, site and production or test scope
MAAS release and delivery	Record version, snap channel and revision, or package version
Controller inventory	Record hostname, role, address, Ubuntu version and failure domain
API and UI endpoint	Record canonical FQDN, HTTPS port, certificate issuer and expiry
Database service	Record endpoint, database name, PostgreSQL major version and HA model
Network scope	Record fabrics, VLAN IDs, CIDRs, gateways and DHCP ownership
Image baseline	Record OS release, architecture, kernel and approved custom image IDs
Configuration repository	Record repository URL, approved revision and change process
Recovery objectives	Record approved RPO, RTO, backup location and last restore test
Secrets and access	Record vault paths, identity groups and break glass procedure

Ownership and escalation

Responsibility	Named owner and backup
MAAS service and on call	Name / team / contact / backup contact
Network and DHCP	Name / team / escalation route
PostgreSQL and recovery	Name / team / escalation route
BMC and physical hardware	Name / team / escalation route
Security and image approval	Name / team / escalation route

Evidence package: attach an inventory export, switch and VLAN map, image manifest, recent successful deployment record, backup manifest and restore exercise report. Keep power parameters, tokens and private keys out of ordinary inventory exports.

Architecture and service boundaries

MAAS coordinates bare metal provisioning through the region and rack controller roles. The region exposes the API and maintains control plane state in PostgreSQL. Rack controllers provide local provisioning and power control connectivity. A combined region and rack installation is valid, but concentrates failure impact. [R02]

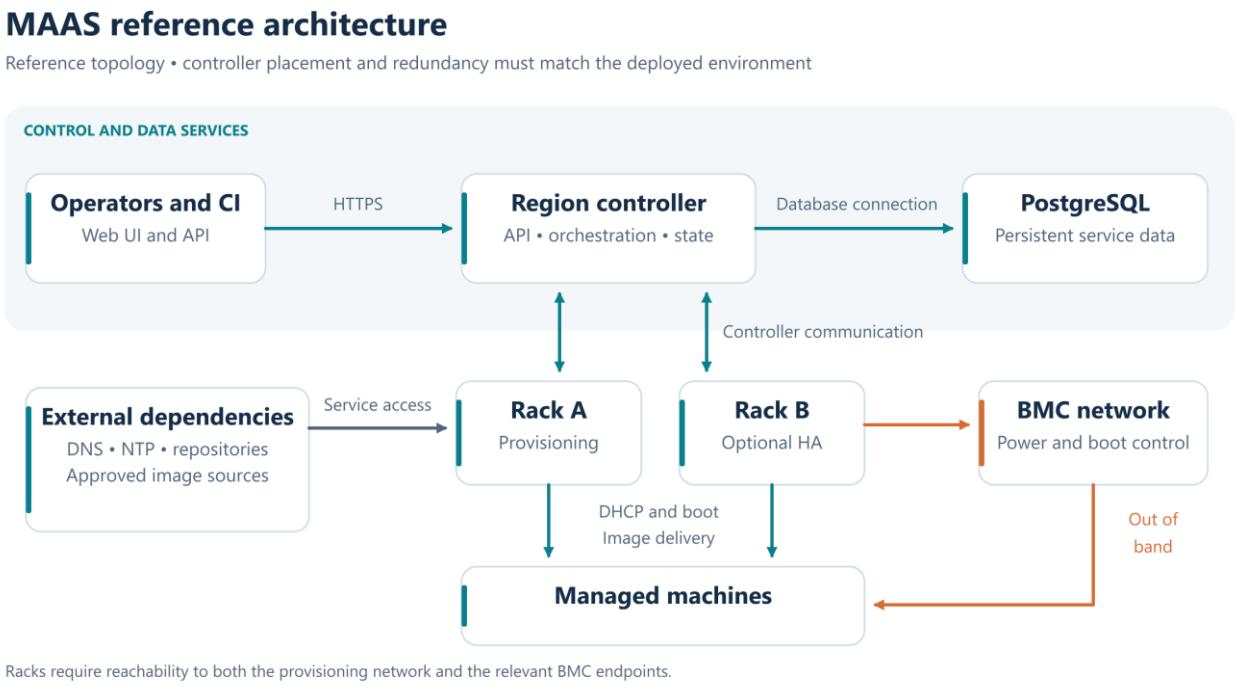


Figure 1. Reference architecture. Map these roles to the production hosts in the configuration register.

Responsibilities across the provisioning path

Component	Operational responsibility
Region and PostgreSQL	Inventory, desired configuration, API operations and durable service state
Rack and provisioning network	Boot services, local image delivery and reachability to managed hardware
BMC network	Authenticated power control and remote console access through the selected driver
External services	DNS, time, package repositories, image sources, monitoring and backup storage

A deployed workload can continue running during a MAAS control plane outage. Provisioning, power operations and any workload dependencies on MAAS hosted DNS, DHCP, proxy or time services can still be affected. Assess those dependencies explicitly before declaring the outage impact.

Keep the management, provisioning and BMC trust boundaries distinct. Document routing, firewall rules and switch redundancy for each boundary. A diagram showing two controllers is a design reference, not proof that failover has been configured or tested.

Release baseline and rebuild procedure

Treat the installed release as the operational baseline. Match Ubuntu, PostgreSQL, packaging, hardware drivers and image support to that release. Snap host compatibility and Debian package dependencies are different checks; use the version specific support notes. [R01]

Capture the baseline before changing anything

```
# On each Ubuntu controller; use the applicable installation method.
cat /etc/os-release
snap list maas                # snap installation
snap info maas                # tracking channel and refresh information
dpkg-query -W 'maas*'         # package installation
maas --version
# On the database host, with the authorized database administrator:
sudo -u postgres psql -Atc 'SHOW server_version;'
```

Selected MAAS release	Published dependency checkpoint
3.5	Ubuntu 22.04 and PostgreSQL 14 in the release matrix
3.6	Ubuntu 24.04; PostgreSQL 14, with 16 recommended
3.7	Ubuntu 24.04 and PostgreSQL 16

These are reference checkpoints, not a selection of the deployed release. Confirm the package or snap notes and supported upgrade path before rebuilding. Record the exact controller and database versions in the recovery manifest.

Controlled rebuild sequence

1. Provision the approved Ubuntu release with stable addressing, DNS, time synchronization and repository access.
2. Install the recorded MAAS release through its approved delivery method. Use a supported production PostgreSQL service and a dedicated MAAS database role.
3. Restrict database listeners and access rules to required controller addresses. Configure authenticated TLS for remote database connections and validate it before initialization.
4. Initialize the intended controller role using the release specific installation guide. For recovery, follow the restore sequence on page 21 instead of creating an empty production service.
5. Create named administrator access, configure HTTPS, import approved SSH public keys and images, then commission and deploy one canary machine.

Use shell assignments such as MAAS_DBUSER=maas without spaces or a leading dollar sign. A database URI contains a secret: inject it using the approved process and avoid logs, shell tracing and shared command histories. Environment variables alone do not eliminate secret exposure. [R03, R04]

Network design and address allocation

Model the physical network in MAAS before assigning machine interfaces. A fabric represents a connected network domain; a VLAN identifies a layer 2 segment; a subnet provides the layer 3 address space. Configuring MAAS objects does not configure physical switch ports. [R05]

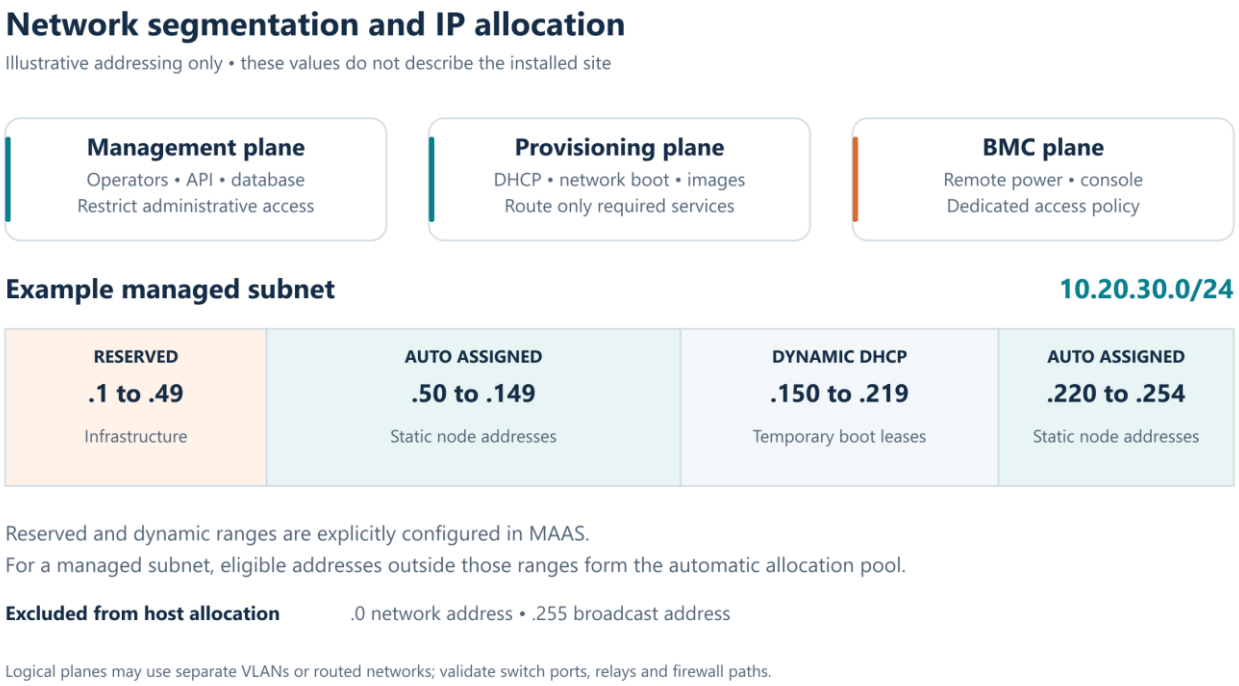


Figure 2. Illustrative address plan. All addresses and VLANs must be replaced by the approved site design.

Address allocation rules

Address category	Use and control
Reserved range	Exclude infrastructure, appliances and externally managed addresses from automatic allocation
Dynamic range	Provide DHCP leases for boot and ephemeral workflows; size for concurrent activity
Auto assignment	Choose free addresses outside reserved and dynamic ranges on a managed subnet
Explicit static address	Assign an approved address to a machine interface and check for external conflicts

Do not confuse an automatically assigned static address with a dynamic DHCP lease. Verify gateway, DNS, MTU, routes, interface mode and VLAN tagging for every boot network. Exclude network and broadcast addresses from IPv4 host allocation. [R06]

For bonds and bridges, document the physical member ports, switch configuration and boot behavior. Validate LACP or active backup behavior on a representative host. Firmware network boot may use a single physical interface before the deployed operating system creates its bond.

DHCP DNS time and firewall dependencies

Establish a single authorized DHCP design for each broadcast domain. For MAAS managed DHCP, configure the VLAN, dynamic range and responsible rack controller. Configure primary and secondary racks explicitly when DHCP redundancy is required. A DHCP relay requires both the physical network relay configuration and the matching MAAS network model. [R07]

Provisioning checks

Confirm firmware boot mode, boot NIC and architecture. Test that the client obtains an address and the correct boot information.

Verify DNS forwarders and internal zones from controllers and machines. Avoid circular forwarding and record reverse DNS ownership.

Use approved time sources. Clock drift can break authentication, certificate validation, logs and incident correlation.

Check repository, proxy and image access from the ephemeral environment as well as from the controller.

Network flow checklist

Source to destination	Typical service	Validation
Operators to API endpoint	HTTPS 443 or configured port	Trusted certificate; route to every intended API backend
Regions to PostgreSQL	TCP 5432 by default	Restricted source addresses; database TLS and authentication
Boot clients and DHCP service	UDP 67/68 for IPv4	VLAN or relay path; no competing replies
Clients to boot services	TFTP UDP 69 and image HTTP	Validate actual rack listeners and related transfer traffic
Clients to resolvers and time	DNS TCP/UDP 53; NTP UDP 123	Only approved service endpoints
Racks to BMCs	Driver dependent	For example Redfish HTTPS or IPMI UDP 623
Controller to controller	Release dependent internal ports	Use the installed release port reference
Controllers to repositories	HTTP/HTTPS or approved proxy	Allow required image and package destinations

This is a flow planning checklist, not a complete firewall ruleset. Native MAAS HTTPS commonly uses port 5443 when configured; internal and boot service ports vary by release. Build the final rules from the release port reference, enabled services and observed listeners. Include return paths, DHCP failover traffic and IPv6 where used. [R08, R09]

Machine lifecycle and operational gates

Use the machine system ID as the stable operational identifier. Confirm hostname, serial number, boot MAC address, BMC address and owner before any state changing action. UI wording can change between releases; the API state and event record are the authoritative operational evidence. [R10]

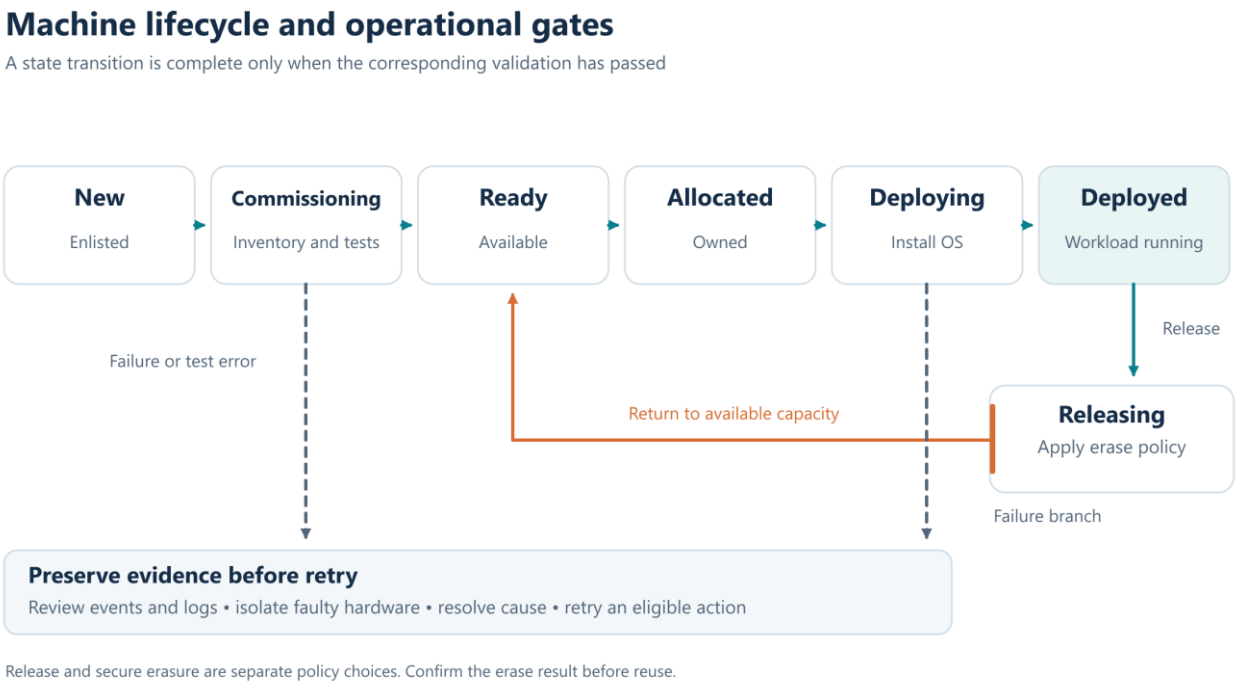


Figure 3. Normal machine lifecycle. Failure states require investigation before a controlled retry.

Gate	Required evidence
New to Commissioning	Hardware identity verified; power control works; boot network is correct
Ready to Allocated	Commissioning results accepted; intended storage and networking reviewed
Allocated to Deploying	Correct owner, image, kernel and user data selected; target confirmed
Deployed to service ready	SSH, cloud-init, network, time, storage and workload checks pass
Releasing to Ready	Workload retired; data retention and erasure decision recorded

Enlistment creates a machine record and collects initial hardware information. Passive network discovery alone does not establish a deployable machine. Recommissioning a Ready machine can refresh its inventory and tests; use supported states and options shown by the installed API. [R11]

Deploy, release, erase and hardware storage changes can destroy data or interrupt service. The operator must have the workload owner approval and verified target identity. Release is not proof of secure erasure; record the chosen erase policy and its result separately.

Commissioning and hardware qualification

Commissioning should produce an accepted hardware inventory and test record. Compare reported CPU, RAM, NICs, disk serials, firmware and boot mode against the asset register. Keep diagnostic evidence before repeating a failed run. Built in scripts and available parameters vary by release. [R12]

Runbook for a new or approved idle machine

1. Validate BMC reachability and credentials from the responsible rack. Confirm console access and boot settings.

2. Review the commissioning script selection and test set. Identify destructive storage tests and firmware or HBA actions before starting.

3. Start commissioning and follow the machine event timeline. Capture script return codes, stdout, stderr and hardware changes.

4. Reconcile LLDP and interface observations with the switch map. Confirm disk identity, capacity and intended boot device.

5. Accept only the expected inventory and passing required tests. Quarantine unexpected hardware or persistent failures.

```
# After authenticated CLI setup on page 11:
maas "$PROFILE" machine read "$SYSTEM_ID"
maas "$PROFILE" machine commission --help
# State changing: only an approved idle machine.
maas "$PROFILE" machine commission "$SYSTEM_ID"
maas "$PROFILE" node-script-results read "$SYSTEM_ID"
```

Custom script engineering

Treat commissioning scripts as privileged code. Review them in source control, pin dependencies, set a bounded timeout and make repeated execution safe. Select target hardware explicitly. Do not assume a numeric filename alone guarantees serialization; supported script metadata controls execution behavior. [R12]

Separate read only inventory from firmware, HBA and RAID changes. Record which custom scripts are selected by default in this deployment.

Use persistent hardware identifiers and explicit preconditions. Fail closed when the controller, disk count or firmware differs from the qualified model.

Avoid general operating system upgrades inside the ephemeral environment. Reboots can discard ephemeral changes; use the supported script reboot behavior when required.

Firmware updates are not an automatic guarantee of commissioning. They occur only through the relevant selected scripts and supported hardware workflow. After a hardware storage change, refresh the MAAS inventory before applying the operating system layout.

Storage layout and RAID operations

Choose one documented storage design for each hardware class. Hardware RAID presents logical disks to the operating system; an HBA exposes physical disks. MAAS and curtin then apply the supported partition, software RAID, LVM and filesystem layout. Validate the boot layout against firmware mode. [R13]

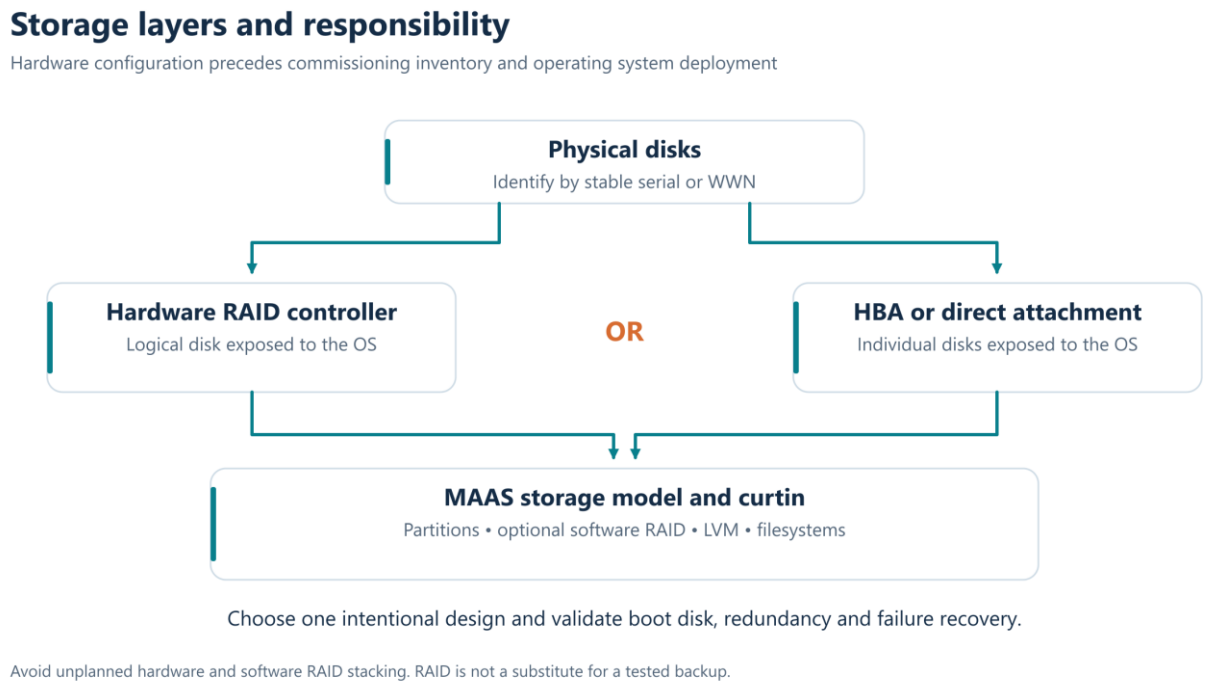


Figure 4. Storage layers. The controller mode determines which devices MAAS can configure.

Read only HPE controller inventory example

```
# Only when the supported HPE SSA CLI is already installed.
sudo ssacli ctrl all show status
sudo ssacli ctrl all show config detail
# On a running Linux host:
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL,SERIAL
findmnt
```

Before changing RAID, record the server serial, controller identity, disk serials, existing logical drives, backup evidence and approved target layout. Use a hardware specific procedure with explicit disk allowlists and a tested rollback or recovery plan. Verify controller health and rescan or recommission after the change.

RAID 0 has no redundancy. Deleting logical drives destroys the existing layout and can make data inaccessible. Do not automate blanket deletion across every detected controller. Do not promise zero downtime: controller firmware, cache state, operating system support and the requested change determine the impact.

For operating system disks, test booting with the intended failure scenario before acceptance. For data disks, specify mount points, ownership, filesystem options, capacity alerts and whether a future deployment is allowed to reformat them.

Deployment runbook and CLI access

Run these examples in Bash on an authorized Ubuntu administration host. Replace the example endpoint and identifiers with approved values. A CLI profile stores the API connection and credentials. Protect that local profile and use an individual or dedicated automation identity. [R14]

```
# Read only connection setup; enter the API key at the prompt.
PROFILE=handover
maas login "$PROFILE" https://maas.example.net/MAAS/api/2.0/
maas "$PROFILE" machines read
SYSTEM_ID=replace_with_verified_system_id
maas "$PROFILE" machine read "$SYSTEM_ID"
maas "$PROFILE" boot-resources read
maas "$PROFILE" machine deploy --help
```

Pre deployment gate

Confirm Ready state, target identity, owner, approved maintenance window and required workload backup.

Review storage, boot disk, interfaces, IP addresses, DNS, routes, kernel and image compatibility.

Validate user data using the target cloud-init version. Confirm all required package repositories are reachable.

```
# State changing: allocates and installs the approved target machine.
# Example image: Ubuntu 24.04; use only if imported and qualified.
maas "$PROFILE" machines allocate system_id="$SYSTEM_ID"
maas "$PROFILE" machine deploy "$SYSTEM_ID" \
  distro_series="ubuntu/noble" \
  user_data="$(base64 -w0 cloud-init.yaml)"
# Observe completion and events; inspect the returned state.
maas "$PROFILE" machine read "$SYSTEM_ID"
maas "$PROFILE" events query id="$SYSTEM_ID"
```

Allocation reserves the machine for the requesting identity. A successful deploy API response means the operation was accepted, not that installation and workload readiness have completed. If a request times out, inspect the state and events before retrying. [R11]

Acceptance and failure handling

After Deployed, verify SSH with the intended key, cloud-init completion, addresses and routes, DNS, time, disks and mounts. Run the workload smoke test. On failure, preserve events and installation logs before redeployment. Correct the first failing stage rather than repeatedly resetting the machine.

Cloud init baseline and validation

Use declarative user data for first boot configuration. Keep host identity, secrets and environment specific settings out of golden images. The following Ubuntu example retains the image default user and uses the SSH public keys managed in MAAS. Verify key access on the canary before applying additional hardening. [R15]

```
#cloud-config
users:
  - default
disable_root: true
ssh_pwauth: false
package_update: true
packages:
  - curl
  - jq
write_files:
  - path: /etc/motd
    owner: root:root
    permissions: '0644'
    content: |
      Managed infrastructure node
      Contact the infrastructure operations team for support.
runcmd:
  - [sh, -c, 'install -d -m 0755 /var/lib/platform']
  - [sh, -c, 'touch /var/lib/platform/bootstrap-complete']
```

Validate before and after deployment

```
# On a validation host with the target cloud-init version:
cloud-init schema --config-file cloud-init.yaml
# After SSH login to the deployed machine:
sudo cloud-init status --wait
sudo cloud-init status --long
sudo tail -n 100 /var/log/cloud-init-output.log
sudo journalctl -u cloud-final --no-pager
```

The completion marker confirms that the last runcmd command ran. It does not replace the cloud-init status, package verification or application health check. Run status --wait from the operator session, not inside a cloud-init command that would wait on itself. Inspect errors and degraded status as well as the return code. [R16]

For explicitly created users, configure approved SSH public keys and lock_passwd: true. Do not embed reusable passwords, private keys or long lived API tokens. Avoid unconditional reboots and broad text replacements in SSH or GRUB files. Kernel parameters, IOMMU settings and distribution specific service names require a separately tested requirement.

Standard images and repository operations

Keep a small approved image catalog with explicit operating system release, architecture and kernel compatibility. Import only the required combinations, and confirm availability before scheduling deployments. Use the stable image stream for the production baseline. [R17]

Image service workflow

1. Record the source URL, trust key configuration, proxy path and approved releases.
2. Select the required image and architecture combinations and start synchronization.
3. Observe import completion and controller disk usage. Verify that the serving rack can deliver the boot resources.
4. Deploy a representative canary and validate installation, boot, network and hardware drivers.
5. Record the accepted image identifiers and keep the previous approved custom artifact available for recovery.

```
maas "$PROFILE" boot-sources read
maas "$PROFILE" boot-resources read
# State changing: imports the already selected resources.
maas "$PROFILE" boot-resources import
maas "$PROFILE" boot-resources is-importing
```

Capacity and disconnected operation

Budget space for downloaded images, custom artifacts, temporary unpacking, logs, database growth and backups. Measure cache behavior and first deployment latency for each rack. Image storage and distribution changed across MAAS releases; capture the actual paths and replication behavior in the backup inventory.

For a restricted network, establish an approved mirror and package repository path. Test DNS, certificate trust, proxy exclusions and access from build hosts, controllers and ephemeral machines. Preserve checksums and trust verification when using local mirrors; do not disable checksum validation to make a build succeed.

Image record	Required value
Identity	Name, resource ID, OS release, architecture and kernel choice
Provenance	Source URL, build revision, artifact hash and package manifest
Qualification	Hardware class, boot mode, test date and acceptance evidence
Lifecycle	Owner, approval, retirement date and previous approved version

Deleting a selected image or changing its source can affect future recovery and deployments. Check consumers, retention obligations and replacement availability before removal. Keep custom image artifacts in durable storage independent of a single controller.

Custom image engineering and promotion

Use a custom image when the required software baseline or installation method justifies maintaining an image pipeline. For small per machine changes, cloud-init and configuration management can reduce image maintenance. Select Canonical templates designed to produce MAAS deployable artifacts. [R18, R19]

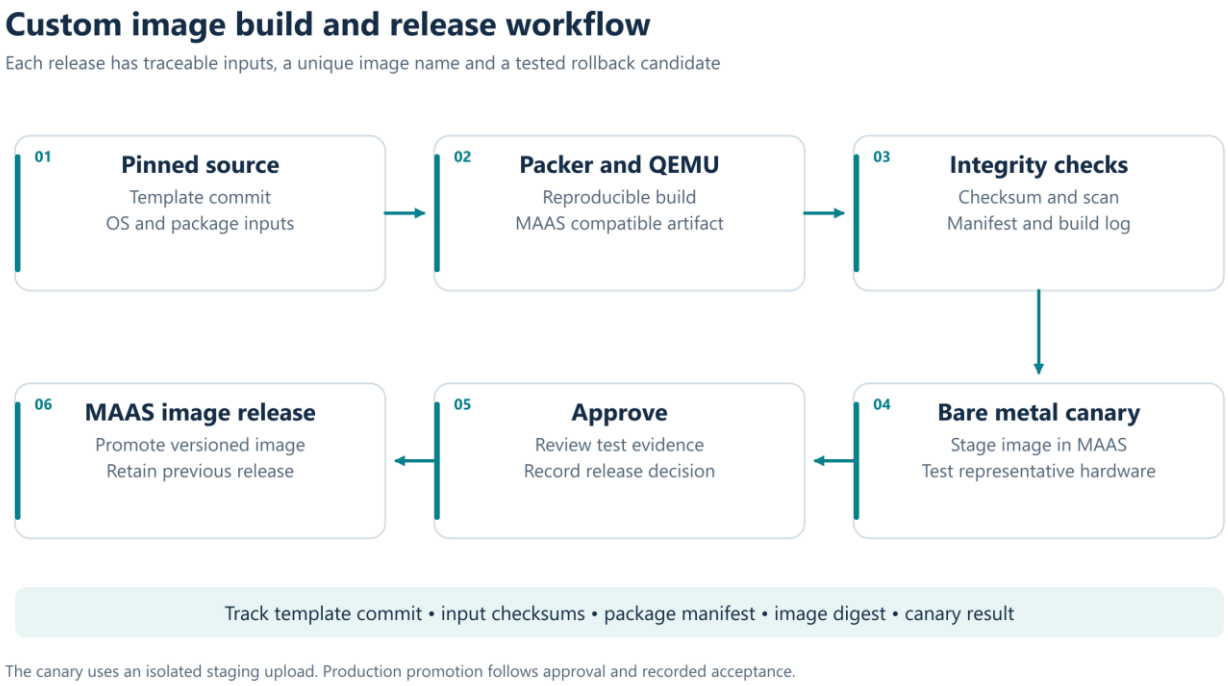


Figure 5. Image promotion pipeline. Each release retains provenance and a tested recovery option.

Reproducible inputs

Pin the packer-maas Git revision, Packer and plugin versions, base image release and checksum, customization script revision and package repository state. A build using moving upstream repositories can change even when its template is unchanged. Store the actual package manifest and artifact hash with every release.

Build boundary and artifact contract

The QEMU builder runs a temporary virtual machine for customization. The template performs the MAAS specific preparation and packaging. An AWS amazon-ebs build produces an AMI and cannot replace this pipeline. An arbitrary ISO or a tar archive of an unrelated filesystem is not a qualified MAAS image.

Promotion gate	Evidence required
Build	Clean completion; pinned inputs; archived build log
Artifact	Expected format; SHA256 recorded; no embedded credentials
Security	Package inventory and vulnerability review with accepted exceptions
Bare metal canary	Boot, storage, network, cloud-init and workload checks pass
Release	Unique versioned name, approver, rollback image and retention record

Test every supported hardware class and relevant firmware mode. QEMU success does not establish that physical NIC, storage, GPU or accelerator drivers work. Include the required kernel and driver compatibility checks in the qualification record.

Build and publish an Ubuntu custom image

Use an isolated build host with hardware virtualization, sufficient temporary disk and controlled repository access. Install Packer from a verified vendor package or verified binary; use a signed-by keyring repository configuration rather than deprecated apt-key. Install the dependencies listed by the selected template revision. [R20]

```
# The exact dependency set belongs to the pinned template README.
git clone https://github.com/canonical/packer-maas.git
cd packer-maas
PACKER_MAAS_REF=replace_with_approved_commit
git checkout --detach "$PACKER_MAAS_REF"
git rev-parse HEAD
cd ubuntu
# Add a reviewed customization script at scripts/site-customize.sh.
packer init .
packer validate -var ubuntu_series=noble \
  -var customize_script=scripts/site-customize.sh .
make custom-cloudimg.tar.gz SERIES=noble \
  CUSTOMIZE=scripts/site-customize.sh
sha256sum custom-cloudimg.tar.gz > custom-cloudimg.tar.gz.sha256
```

This example follows the Ubuntu template interface. Confirm the Makefile targets, variables, privileges and dependencies at the approved revision. The cloud image output is a filesystem archive; a raw disk output uses a different format and upload filetype. [R19]

Publish a uniquely named candidate

```
maas "$PROFILE" boot-resources create --help
# Upload only after local artifact validation.
maas "$PROFILE" boot-resources create \
  name='custom/ubuntu-noble-site-v001' \
  title='Ubuntu Noble Site v001' \
  architecture='amd64/generic' \
  base_image='ubuntu/noble' \
  filetype='tgz' \
  content@=custom-cloudimg.tar.gz
maas "$PROFILE" boot-resources read
```

Match base_image to the actual Ubuntu base and verify support in the installed API. For a compressed raw disk artifact use the corresponding supported type, such as ddgz, rather than tgz. The CLI upload behavior is version sensitive; use its local help and the matching boot resource documentation. [R21]

Select the uploaded custom image on an allocated canary and complete the acceptance checks before production promotion. Record its resource ID, hash and deployment evidence. Keep image names immutable by release; a rollback selects a previous tested artifact and may require reinstalling the target machine.

Access control and security operations

MAAS holds infrastructure inventory and credentials capable of powering and reinstalling servers. Restrict its administrative access to the management network and approved operator identities. Make account ownership and credential rotation part of the handover.

Control area	Required operating practice
Identity	Named users; separate automation identities; prompt removal of departed users
Authorization	Minimum supported privileges; verify resource pool and group enforcement for the release
API and UI transport	Trusted HTTPS; correct hostname and chain; expiry monitoring
API credentials	Secret store reference, rotation owner and revocation procedure
BMC access	Dedicated accounts, isolated network, driver compatible privileges and certificate policy
Database	Dedicated role, narrow source rules, encrypted remote transport and protected backups
Privileged scripts	Peer review, pinned dependencies, controlled upload and execution
Audit evidence	Central retention of identity, configuration and state change events

Database connection example

```
# Illustrative pg_hba.conf record, not a complete database configuration.
# Substitute the actual database, role and single controller address.
hostssl  maas  maas  10.10.10.11/32  scram-sha-256
```

hostssl applies only when PostgreSQL SSL is enabled. Validate client certificate verification and connection options supported by the deployed MAAS release. HBA rules are evaluated in order; remove or narrow conflicting broad allow rules. Check the effective rule and a successful authenticated connection. [R04]

Native MAAS TLS configuration is available in documented releases from 3.3 onward; the default configured HTTPS port is 5443. If a reverse proxy or load balancer provides HTTPS, document backend transport, trust boundaries and the canonical URL. Do not use routine insecure certificate bypasses. [R09]

Authorization features change across releases. Resource organization alone is not proof of enforced isolation. Test an ordinary operator account against the intended allow and deny cases. Keep a controlled, audited emergency access path and test credential recovery without exposing secrets in this document.

Automation and configuration management

Automate against the installed API description and a reviewed configuration repository. Use stable system IDs, explicit image versions and validated preconditions. Keep desired configuration, execution evidence and secrets in their respective controlled stores.

Export a minimal operational inventory

```
# Read only. jq must be available on the administration host.
umask 077
maas "$PROFILE" machines read | jq '
  [.[] | {system_id, hostname, status_name, architecture,
          osystem, distro_series}]' > machine-inventory.json
maas "$PROFILE" subnets read > subnet-inventory.json
maas "$PROFILE" boot-resources read > image-inventory.json
# After an approved server upgrade:
maas refresh
```

Review exports before sharing them: addresses, hostnames and infrastructure relationships can be sensitive even when credentials are absent. A JSON inventory export helps audit configuration but is not a complete recoverable backup. CLI profiles cache credentials and API descriptions; refresh updates the latter. [R14, R22]

Reliable workflow behavior

Resolve and validate the intended machine before sending an action. Require the expected state and ownership; reject ambiguous matches.

Treat state changing operations as asynchronous. Poll for a terminal state with a timeout and retain the operation timeline.

On transport failure, inspect current state before retrying. Blind retries can duplicate work or act after the original request succeeded.

Serialize conflicting changes to the same machine, image or network object. Stop the workflow when a required gate fails.

Keep deployment user data and commissioning scripts versioned. Capture their revision with the machine deployment record.

Configuration ownership

Use MAAS for the supported bare metal and initial network/storage configuration. Assign ongoing operating system and application policy to the chosen configuration management system. Document who owns each setting so subsequent automation does not overwrite the network or storage design unexpectedly.

A release record should join the change ticket, image resource ID and checksum, script revision, user data revision, target system IDs and acceptance evidence. This makes a failed rollout diagnosable and supports a controlled recovery.

Monitoring and service objectives

Monitor user visible provisioning outcomes and the dependencies that explain them. Record the agreed service objectives in the configuration register. The thresholds below are proposed starting points for local tuning, not measured performance or a service commitment.

Signal	Initial alert condition	Response
API and controllers	Endpoint unavailable or persistent errors	Check controller, database and network health
Provisioning success	Failure rate exceeds the agreed baseline	Group by image, rack, hardware class and failure stage
Stalled operations	Duration exceeds the qualified workflow limit	Inspect events and the first incomplete stage
Dynamic address capacity	Less than 20 percent free	Review active leases and provisioning concurrency
Controller disk capacity	Less than 20 percent free	Check image growth, logs and backup staging
Database	Connection, storage or replication threshold exceeded	Escalate to database owner before changing limits
Certificate expiry	Less than 30 days remaining	Renew and test the complete certificate chain
Backup age	Older than the approved recovery schedule	Investigate job and off host copy completion

Exporter and log integration

The monitoring guide documents region metrics on port 5239, rack metrics on 5249 and optional cluster statistics through the MAAS HTTP service. Confirm the enabled endpoints and actual metrics for the installed release, restrict collector access and validate labels before writing alerts. [R23]

```
# Discover actual unit names before using a release specific log command.
systemctl list-units --type=service | grep -E 'maas|postgres'
# Example for current snap layouts:
sudo journalctl -u snap.maas.pebble -t maas-regiond --since '-30m'
sudo journalctl -u snap.maas.pebble -t maas-rackd --since '-30m'
# Package layout alternatives:
sudo journalctl -u maas-regiond --since '-30m'
sudo journalctl -u maas-rackd --since '-30m'
```

Centralize logs with synchronized timestamps and an agreed retention period. Track deployment duration by stage and hardware class. Test alert routing and acknowledgment with the receiving team; an installed dashboard without a responding owner is not an operational control. [R24]

Troubleshooting by failure stage

Locate the first failed transition. Preserve the system ID, serial, UTC timestamps, selected image, recent changes, event timeline and script output before retrying. Compare the affected host with a known good host on the same rack and network.

Observed symptom	First checks	Useful evidence
Cannot power on	BMC route, account, driver and firmware	Rack logs; BMC event log; remote console
No DHCP lease	Boot NIC, VLAN, switch port and relay	DHCP capture; lease state; competing server replies
Boot file unavailable	Firmware architecture and boot service path	Console error; TFTP/HTTP reachability; image availability
Commissioning fails	First failed script and hardware compatibility	Return code; stdout/stderr; inventory differences
Installation fails	Image, disk layout, repository and network	Curtin log; storage errors; installer console
Deployed but no SSH	Assigned address, route, key and host firewall	Console; SSH client diagnostics; cloud-init logs
Cloud-init incomplete	YAML, package access and command failures	cloud-init status; output log; cloud-final journal
Intermittent failures	MTU, packet loss, DNS/time and concurrency	Correlated timestamps; controller and database metrics

Read only diagnostic commands

```
maas "$PROFILE" machine read "$SYSTEM_ID"
maas "$PROFILE" events query id="$SYSTEM_ID"
maas "$PROFILE" node-script-results read "$SYSTEM_ID"
# On the affected Linux host, if accessible:
ip -br address
ip route
resolvectl status
timedatectl status
lsblk -f
sudo cloud-init status --long
```

For an installation failure, inspect the MAAS installation output and curtin logs in the relevant ephemeral or target environment. Capture the console when networking is unavailable. Script output returned by the API may be base64 encoded; decode it before analysis. [R11, R24]

Avoid restarting every component at once. Make one justified change, repeat the smallest failing test and record the outcome. Escalation should include the first failing stage, scope, timeline, evidence and attempted changes, with credentials removed.

Backup design and capture procedure

Define one recovery set containing database state, application configuration and the artifacts required to recreate service behavior. A snap snapshot does not replace an external PostgreSQL backup. Agree the RPO, RTO, retention, encryption and off host storage location with the service owner. [R25]

Backup scope	Recovery evidence
PostgreSQL	Database dump or approved physical backup; roles; version; recovery instructions
MAAS application state	Snap snapshot or package configuration and applicable state directories
Secrets and TLS	Protected secret references or recoverable encrypted material; recovery access
Images and scripts	Custom artifacts, checksums, build sources, commissioning and deployment scripts
Infrastructure dependencies	Network, DNS, proxy, monitoring and controller build configuration

Coordinated capture runbook

Drain active provisioning and freeze configuration changes. Stop MAAS writers on all relevant controllers during the coordinated capture window. Keep PostgreSQL available for a logical dump. PostgreSQL can produce a consistent database dump while running; the change freeze also aligns the application files and external artifacts. [R26]

```
# Examples: run with approved access on the correct respective hosts.
umask 077
BACKUP_DIR=/approved/backup/location/recovery-set
mkdir -p "$BACKUP_DIR"
# On each snap controller, after draining work:
sudo snap stop maas
# On a local PostgreSQL host using the database superuser:
sudo -u postgres pg_dump -Fc maas > "$BACKUP_DIR/maas.dump"
sudo -u postgres pg_dumpall --globals-only \
  > "$BACKUP_DIR/postgres-globals.sql"
# On each snap controller; record the returned snapshot set ID:
sudo snap save maas
sudo snap check-snapshot SNAPSHOT_SET_ID
sudo snap export-snapshot SNAPSHOT_SET_ID \
  "$BACKUP_DIR/maas-snapshot.zip"
sudo snap start maas
```

Use the actual database name and deployment specific authentication. For remote PostgreSQL, use the DBA approved connection method. Check every exit code; global dumps can contain password hashes. Copy the complete recovery set to encrypted off host storage, record checksums and restart all stopped controllers even if capture fails. [R27]

For package installations, archive /etc/maas and the required /var/lib/maas state using the release specific procedure. Record any excluded stock images and their re-download time. A successful backup job is accepted only after restore validation.

Disaster recovery and restore validation

Restore first on an isolated network. Prevent restored controllers from reaching production BMCs or serving competing DHCP. Recovery authority must cover database, network, secrets and hardware control; keep those contacts available outside the MAAS service.

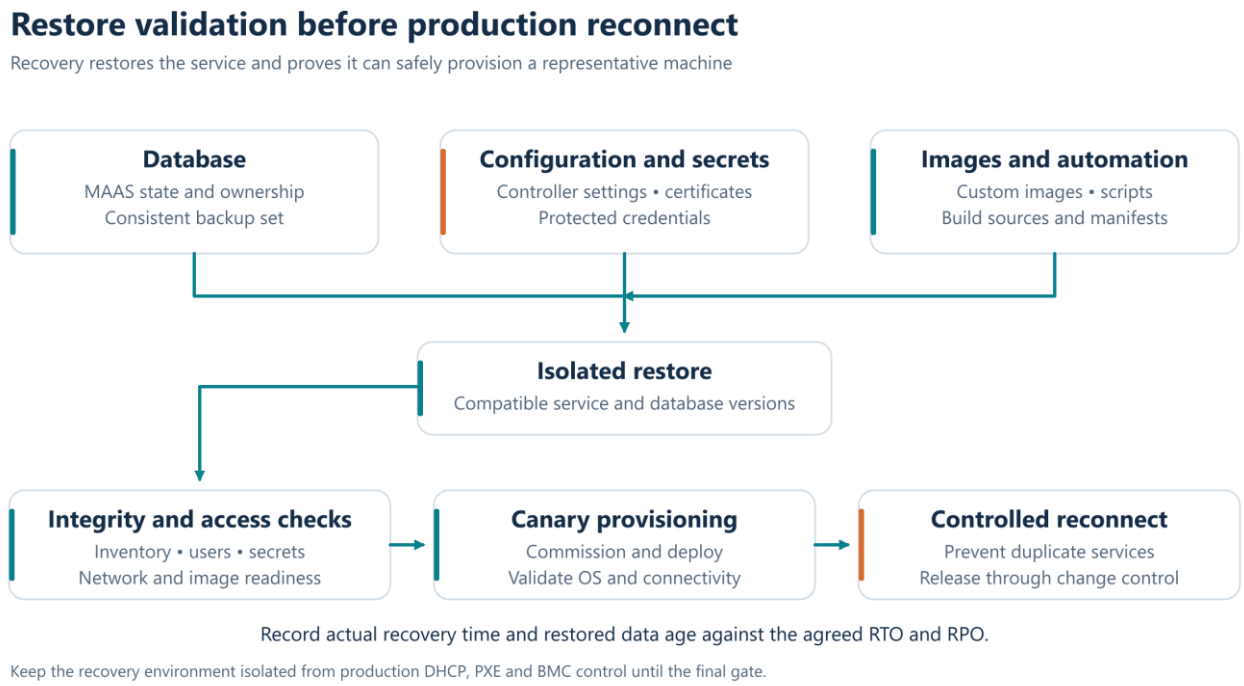


Figure 6. Recovery sequence. Production reconnection follows evidence based acceptance.

Restore sequence

1. Declare the recovery point and scope. Verify backup integrity, credentials, controller versions and target isolation.
2. Rebuild compatible Ubuntu and PostgreSQL hosts. Restore database roles and data through the DBA approved procedure. Validate ownership, extensions and application access.
3. Install the appropriate MAAS version and stop its services. Restore application state, configuration, keys and custom artifacts from the same recovery set.
4. For snap state, import the exported snapshot and use the newly returned set ID when restoring. Do not assume the original snapshot ID survives import. [R27]
5. Start services in isolation. Validate inventory, controller registration, networks, image availability, HTTPS, identity access and audit behavior.
6. Deploy and release an approved canary on the isolated provisioning network. Measure elapsed recovery time and identify any missing dependencies.
7. Reconnect production deliberately, verifying DHCP ownership and BMC reachability before restoring normal operations.

Do not combine a migrated database with an older MAAS binary unless the release documentation explicitly supports it. A complete recovery point must include compatible application and database state. Downgrading a snap alone is not a general database rollback strategy.

Record actual data loss relative to the selected recovery point, total restoration time, canary evidence, exceptions and owner acceptance. Repeat the drill after material changes to packaging, database version, network topology or secret storage.

Availability and controlled upgrades

Availability is a property of the whole path

For a redundant design, provide multiple region controllers, redundant racks serving the required networks, a resilient API entry point and a separately managed PostgreSQL HA service. Configure DHCP primary and secondary racks on each applicable VLAN. Identify shared switches, power, storage, DNS and image sources that can still fail together. [R28]

Failure test	Expected evidence
One rack unavailable	Approved boot and power workflows continue through the surviving path
One region unavailable	API access and provisioning remain functional through healthy backends
Database failover	Application reconnects correctly; no unauthorized dual primary operation
DHCP primary unavailable	New test client obtains a valid lease through the configured failover design
Upstream dependency lost	Known impact and recovery procedure; appropriate alert reaches the owner

Upgrade runbook

1. Review the exact source and target release notes, Ubuntu and PostgreSQL compatibility, supported upgrade path and known issues.
2. Rehearse the change and recovery in a representative staging environment. Test custom scripts, image import, API clients and a bare metal canary.
3. Freeze changes, drain active work and capture a complete verified recovery set. Confirm operator and DBA availability.
4. Follow the release specific controller order and maintenance requirements. The current upgrade guide directs rack upgrades before regions; verify the applicable path. [R29]
5. Refresh CLI API descriptions, validate service health and repeat the deployment acceptance tests.
6. Record resulting revisions, schema state, timings, issues and owner acceptance; reopen normal operations.

The current upgrade guide states that MAAS 3.7.4 and later cannot communicate across controller versions. Plan all controller upgrades within the same maintenance window for the applicable transition; do not assume mixed versions provide a rolling upgrade guarantee. [R29]

Set an explicit snap refresh or package maintenance policy. Preserve a supported recovery path before database migration. If validation fails, freeze further changes and use the rehearsed recovery procedure rather than improvising a binary downgrade.

Routine operations and capacity management

Assign recurring tasks to named owners and retain evidence in the operations record. Tune the cadence to service criticality and fleet change rate. Review the runbook after incidents and after each material release change.

Cadence	Operational checks	Owner
Daily	Controller and API health; failed or stalled operations; backup completion; disk and lease capacity	On call operator
Weekly	Image synchronization; hardware failures; growth trends; alert quality; open exceptions	MAAS service owner
Monthly	Access and token review; certificate horizon; image vulnerability review; capacity forecast	Service and security owners
Quarterly or after major change	Isolated restore exercise; failover tests; canary across hardware classes; handover register review	Service, database and network owners
Before each rollout	Approved image and scripts; target identity; data retention; maintenance and recovery plan	Change owner

Capacity planning

Size for simultaneous commissioning and deployment, not only total machine count. Measure region CPU and memory, database latency and connections, rack serving throughput, boot network utilization, dynamic address headroom and image disk growth. Maintain a representative concurrency test with pass and stop criteria.

Estimate image storage from the retained releases and architectures, custom image versions, temporary build or import workspace and recovery copies. Record both normal and peak usage. Set capacity thresholds before a full disk blocks imports or disrupts the database.

Retirement and data handling

Before releasing or retiring a machine, obtain workload owner approval, complete required backups, revoke workload access and select the approved erasure policy. Verify the erasure result separately from the MAAS state transition. Remove stale DNS, monitoring and inventory records only after confirming ownership and retention requirements.

For image retirement, identify dependent deployment procedures and recovery plans before deletion. For controller retirement, transfer network service responsibility and verify the surviving controller paths before shutdown. Record the change in the architecture and configuration register.

Acceptance tests and operational signoff

Complete the tests on approved test hardware and retain the evidence location. A passing status must represent an observed result. Resolve critical failures before ownership transfer or obtain explicit acceptance of the remaining risk.

ID and test	Pass criterion	Result and evidence
AT01 Access	Named operator can log in over trusted HTTPS; unauthorized access is denied	Record result / link
AT02 Inventory	Serials, BMCs, boot MACs and network topology match the asset register	Record result / link
AT03 Commission	Required hardware inventory and tests pass on each supported class	Record result / link
AT04 Deploy	Approved image reaches Deployed and completes host acceptance checks	Record result / link
AT05 Image pipeline	Custom artifact has provenance, checksum and bare metal canary evidence	Record result / link
AT06 Network	DNS, routes, MTU, DHCP and time work from boot and deployed environments	Record result / link
AT07 Storage	Expected layout, mounts and supported failure behavior are verified	Record result / link
AT08 Monitoring	A test alert reaches the owner; relevant evidence can be retrieved	Record result / link
AT09 Recovery	Isolated restore and canary complete within the approved RPO and RTO	Record result / link
AT10 Availability	Configured controller, DHCP and database failover scenarios pass	Record result / link
AT11 Lifecycle	Approved release and erasure workflow behaves as documented	Record result / link
AT12 Knowledge transfer	Receiving team independently performs deployment and diagnosis	Record result / link

Transfer record

Approval field	Record
Delivering owner and date	Name / date / evidence package revision
Receiving owner and date	Name / date / operational access verified
Open exceptions	Issue / impact / accountable owner / due date
Acceptance decision	Accepted / accepted with recorded exceptions / pending

Attach the completed configuration register, tested recovery procedure, approved image manifest, access references and escalation contacts. Schedule the first review with the receiving team and record the support transition date.

Operator quick reference and terminology

Start of shift

Check service health, failed operations, current incidents and the change calendar.

Confirm the most recent off host backup and available capacity.

For each requested action, verify target identity, ownership, expected state and approval.

Read only command reference

```
maas "$PROFILE" machines read
maas "$PROFILE" machine read "$SYSTEM_ID"
maas "$PROFILE" events query id="$SYSTEM_ID"
maas "$PROFILE" node-script-results read "$SYSTEM_ID"
maas "$PROFILE" boot-resources read
maas "$PROFILE" subnets read
maas "$PROFILE" rack-controllers read
maas "$PROFILE" region-controllers read
```

Term	Operational meaning
BMC	Out of band management controller used for power control and console access
Enlistment	Initial registration and identification of a machine in MAAS
Commissioning	Hardware discovery and selected qualification or configuration scripts
Curtin	Installation component used to apply the target operating system and layout
Cloud-init	First boot initialization using metadata and user data
CLI profile	Saved API endpoint, credentials and API description for a MAAS connection
Resource pool and zone	Machine organization and placement constructs; authorization depends on release and configuration
RPO and RTO	Maximum accepted data loss interval and target time to restore service
Canary	A small representative deployment used to validate a change before expansion

Command names and available options are resolved against the installed server. Use maas "$PROFILE" --help and operation specific --help when validating a procedure. Treat examples with replacement values as templates and confirm every state changing command before execution.

Technical references

Primary documentation checked on 24 October 2026. Select the documentation version matching the installed service. The latest documentation may include capabilities absent from older deployments.

R01 Supported MAAS versions

https://canonical.com/maas/docs/release-information/supported-maas-versions/

R02 Controllers

https://canonical.com/maas/docs/latest/explanation/controllers/

R03 Install MAAS

https://canonical.com/maas/docs/latest/how-to-guides/get-started/install-maas/

R04 PostgreSQL client authentication

https://www.postgresql.org/docs/16/auth-pg-hba-conf.html

R05 MAAS networking

https://canonical.com/maas/docs/latest/explanation/networking/

R06 MAAS network address ranges

https://certification.canonical.com/docs/programmes/server/Environment_Setup_Guide/appendix-c-maas-network-range/

R07 Manage network services

https://canonical.com/maas/docs/stable/how-to-guides/manage-network-services/

R08 MAAS 3.7 configuration and port tables

https://canonical.com/maas/docs/3.7/reference/configuration-guides/configuration-tables/

R09 Enhance MAAS security

https://canonical.com/maas/docs/latest/how-to-guides/enhance-maas-security/

R10 Machine lifecycle

https://canonical.com/maas/docs/3.7/explanation/the-machine-life-cycle/

R11 Machine CLI reference

https://canonical.com/maas/docs/latest/reference/cli-reference/machine/

R12 Commissioning scripts

https://maas.io/docs/reference-commissioning-scripts

R13 Storage reference

https://canonical.com/maas/docs/latest/reference/configuration-guides/storage/

R14 CLI login reference

https://canonical.com/maas/docs/latest/reference/cli-reference/login/

R15 Cloud-init module reference

https://docs.cloud-init.io/en/latest/reference/modules.html

Technical references continued

Primary documentation checked on 24 October 2026. Select the documentation version matching the installed service. The latest documentation may include capabilities absent from older deployments.

R16 Wait for cloud-init

https://docs.cloud-init.io/en/latest/howto/wait_for_cloud_init.html

R17 Manage images

https://canonical.com/maas/docs/3.7/how-to-guides/manage-images/

R18 MAAS image concepts

https://canonical.com/maas/docs/latest/explanation/images/

R19 Canonical Ubuntu Packer templates

https://github.com/canonical/packer-maas/blob/main/ubuntu/README.md

R20 Install and verify Packer

https://developer.hashicorp.com/packer/install

R21 Boot resource CLI reference

https://canonical.com/maas/docs/latest/reference/cli-reference/boot-resource/

R22 Refresh CLI API descriptions

https://canonical.com/maas/docs/refresh

R23 Monitor MAAS

https://canonical.com/maas/docs/latest/how-to-guides/monitor-maas/

R24 Use logging

https://canonical.com/maas/docs/latest/how-to-guides/use-logging/

R25 Back up MAAS

https://canonical.com/maas/docs/latest/how-to-guides/back-up-maas/

R26 PostgreSQL SQL dump backups

https://www.postgresql.org/docs/16/backup-dump.html

R27 Snap data snapshots

https://snapcraft.io/docs/how-to-guides/manage-snaps/create-data-snapshots/

R28 Manage high availability

https://canonical.com/maas/docs/latest/how-to-guides/manage-high-availability/

R29 Upgrade MAAS

https://canonical.com/maas/docs/latest/how-to-guides/upgrade-maas/

Document baseline

MAAS Tutorials, supplied project reference, 37 pages. This edition retains its operational topics and adds release qualification, image promotion, security, monitoring, recovery and handover acceptance procedures.